

# Xtream API Protocol

## Comprehensive Technical Documentation & Reference Manual

### 1. Overview

---

The **Xtream API protocol** is a query-string-based REST-like interface natively used by IPTV/OTT media players (including TiviMate, IPTV Smarters, and GSE Smart IPTV) to dynamically fetch data structures from media streaming panels and servers.

Unlike flat standard .m3u playlists—which force client engines to download and parse massive text files potentially holding millions of characters all at once—the Xtream architecture isolates data queries into localized, on-demand HTTP GET operations. This structural abstraction allows client software platforms to seamlessly paginate category nodes, pull rich asset descriptions asynchronously, and precisely update server-side access privileges without reloading global manifests.

### 2. Global Endpoint Architecture

---

All connectivity routines initiated with an Xtream-compatible server endpoint demand three localized credential components provided by an operator infrastructure:

- **Base URL:** The primary target host machine domain or IP assigned with an active network port (e.g., `http://example.com:8080`)
- **Username:** The profile credential token (<username>)
- **Password:** The security access token (<password>)

The protocol abstracts routing routines into three standard access scripts deployed at the root level of the server environment:

| Target Script   | Functional Pipeline Objective                                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /player_api.php | The principal transactional routing layer. Resolves validation, handles multi-tier catalog generation, and returns target streaming nodes via specified action keys. |
| /xmltv.php      | Compiles and streams Electronic Program Guide (EPG) chronologies natively formatted inside structural XML schemas.                                                   |
| /get.php        | A legacy fallback pipeline translating internal databases into traditional flat text M3U formatting on-the-fly.                                                      |

### 3. Authentication & Account Profiling

To register a client runtime device, parse initial server connection availability, and fetch core user account properties, the interface demands an initial validation request aimed directly at `player_api.php` without specifying an explicit action modifier block.

#### Request Structure

```
GET http://<base_url>/player_api.php?username=<username>&password=<password>
```

#### Structural Payload Mapping

The server replies with a root JSON object strictly separated into `user_info` status models and localized `server_info` configuration trees:

```

{
  "user_info": {
    "username": "john_doe",
    "password": "securepass123",
    "message": "Hello John Doe, welcome back!",
    "auth": 1,
    "status": "Active",
    "exp_date": "1782297600",
    "is_trial": "0",
    "active_cons": "1",
    "max_connections": "2",
    "created_at": "1609459200",
    "allowed_output_formats": [
      "ts",
      "mkv",
      "m3u8"
    ]
  },
  "server_info": {
    "url": "example.com",
    "port": "8080",
    "https_port": "8443",
    "server_protocol": "http",
    "rtmp_port": "1935",
    "timezone": "Europe/London",
    "timestamp": 1719220000
  }
}

```

## 4. Live TV Engine

---

Live TV interactions split category taxonomy mappings away from the literal logical media channels. This design pattern protects low-powered systems from memory leaks and heavy runtime calculation overhead.

## Action Parameter References

| Action Tag          | Description                                                       | Required Parameters |
|---------------------|-------------------------------------------------------------------|---------------------|
| get_live_categories | Returns a listing of all available Live TV categories.            | None                |
| get_live_streams    | Fetches the full global array of every authorized Live stream.    | None                |
| get_live_streams    | Isolates live streaming objects strictly to an explicit category. | &category_id=<ID>   |

## Live Parsing Code Blueprint Examples

To grab target structural live category maps:

```
GET http://<base_url>/player_api.php?
username=<username>&password=<password>&action=get_live_categories
```

```
[
  {
    "category_id": "10",
    "category_name": "UK News",
    "parent_id": 0
  }
]
```

To request stream nodes bound to Category ID 10:

```
GET http://<base_url>/player_api.php?
username=<username>&password=<password>&action=get_live_streams&category_id=10
```

```
[
  {
    "num": 1,
    "name": "Sky News HD",
    "stream_type": "live",
    "stream_id": 4122,
    "stream_icon": "http://example.com/logos/skynews.png",
    "epg_channel_id": "SkyNews.uk",
    "added": "1609459200",
    "category_id": "10",
    "custom_sid": "",
    "tv_archive": 1,
    "tv_archive_duration": 7
  }
]
```

## 5. Video On Demand (VOD) Engine

The Video on Demand metadata system targets individual cinematic resources, augmenting raw video addresses with rich indexing strings, container properties, and data parsed directly from structural repositories such as TMDb or IMDb.

### Action Parameter References

| Action Tag         | Description                                                        | Required Parameters          |
|--------------------|--------------------------------------------------------------------|------------------------------|
| get_vod_categories | Extracts listing identifiers of localized movie categories.        | None                         |
| get_vod_streams    | Retrieves movies (global or category-filtered list models).        | &category_id=<ID> (Optional) |
| get_vod_info       | Pulls deep cryptographic metadata properties for one single movie. | &vod_id=<ID>                 |

### VOD Information Retrieval Sample

```
GET http://<base_url>/player_api.php?
username=<username>&password=<password>&action=get_vod_info&vod_id=8831
```

```
{
  "info": {
    "tmdb_id": "299534",
    "imdb_id": "tt4154796",
    "name": "Avengers: Endgame",
    "o_name": "Avengers: Endgame",
    "cover_big": "http://example.com/covers/endgame.jpg",
    "movie_image": "http://example.com/covers/endgame.jpg",
    "release_date": "2019-04-26",
    "episode_run_time": "181",
    "youtube_trailer": "TcMBFSGVi1c",
    "director": "Anthony Russo, Joe Russo",
    "actors": "Robert Downey Jr., Chris Evans, Mark Ruffalo",
    "description": "After the devastating events of Avengers: Infinity War...",
    "plot": "After the devastating events of Avengers: Infinity War...",
    "age": "PG-13",
    "mpaa_rating": "PG-13",
    "rating": "8.4",
    "duration_secs": 10860,
    "duration": "03:01:00",
    "genres": [
      "Action",
      "Sci-Fi",
      "Adventure"
    ]
  },
  "movie_data": {
    "stream_id": 8831,
    "name": "Avengers: Endgame",
    "added": "1609459200",
    "category_id": "20",
    "container_extension": "mp4",
    "custom_sid": null,
    "direct_source": ""
  }
}
```

## 6. TV Series Engine

Because serial television requires a multi-tiered parent-to-child data model (**Series** → **Season** → **Episode**), the API segregates the overarching series shell metadata entry from the nested episodic run files.

## Action Parameter References

| Action Tag            | Description                                                        | Required Parameters          |
|-----------------------|--------------------------------------------------------------------|------------------------------|
| get_series_categories | Pulls down structural lists for TV Show group mappings.            | None                         |
| get_series            | Returns a summary block listing of overall parent series packages. | &category_id=<ID> (Optional) |
| get_series_info       | Extracts multi-season blueprints and individual episode streams.   | &series_id=<ID>              |

### Deep Manifest Serialization (get\_series\_info)

```
GET http://<base_url>/player_api.php?
username=<username>&password=<password>&action=get_series_info&series_id=502
```

```
{
  "seasons": [
    {
      "air_date": "2008-01-20",
      "episode_count": 7,
      "id": 1,
      "name": "Season 1",
      "season_number": 1
    }
  ],
  "info": {
    "name": "Breaking Bad",
    "cover": "http://example.com/covers/bb.jpg",
    "plot": "A high school chemistry teacher diagnosed with lung cancer...",
    "director": "Vince Gilligan",
    "genre": "Drama, Crime",
    "rating": "9.5"
  },
  "episodes": {
    "1": [
      {
        "id": "102451",
        "episode_num": 1,
        "title": "Pilot",
        "container_extension": "mkv",
        "info": {
          "duration": "00:58:00",
          "duration_secs": 3480
        },
        "season": 1
      }
    ]
  }
}
```

## 7. EPG Engine (Electronic Program Guide)

To capture timeline metrics and schedule mappings accurately, the Xstream layout supplies alternative routing structures to handle chronological parsing demands based on system requirements:

### 1. The Monolithic XMLTV Pipeline

This path returns the global system schedule block, heavily compressed and formatted into standard standard xml tags. It is ideal for players executing deep visual grid pre-rendering actions.

**GET** [http://<base\\_url>/xmltv.php?username=<username>&password=<password>](http://<base_url>/xmltv.php?username=<username>&password=<password>)



## 2. The Micro Short-EPG Timeline Query

Pulls an exceptionally thin, single-channel structural array tracking immediate schedule intervals (e.g., current program and subsequent show listings).

```
GET http://<base_url>/player_api.php?
username=<username>&password=<password>&action=get_short_epg&stream_id=4122&limit=5
```

## 8. Stream URL Construction

Once raw metadata maps are processed via JSON arrays, the client media application does not ping the server interface to compute discrete stream files. Instead, playing nodes are programmatically formatted client-side via clear structural routing tokens:

### Live Streams Path Rule

Applies a distinct `/live/` URL locator token. Extensions must mirror structural entries permitted by user profiles (natively `ts` or HLS `m3u8` structures):

```
http://<base_url>/live/<username>/<password>/<stream_id>.<extension>
```

*Production Form Target:* `http://example.com:8080/live/john_doe/securepass123/4122.ts`

### VOD (Movies) Path Rule

Deploys a distinct `/movie/` URL tracking component tied to the resource's structural file extension metadata indicator:

```
http://<base_url>/movie/<username>/<password>/<vod_id>.<extension>
```

*Production Form Target:* `http://example.com:8080/movie/john_doe/securepass123/8831.mp4`

### TV Series Episodes Path Rule

Routes targets through a distinct `/series/` namespace indicator. **Critical Directive:** The structural payload key placed at the absolute terminal path tail must explicitly be the unique internal **episode tracking resource identifier ID** extracted out from inside the nested episodes array block, *not* the collection's generic parental `series_id` value.

```
http://<base_url>/series/<username>/<password>/<episode_id>.<extension>
```

*Production*                      *Form*                      *Target:*                      `http://example.com:8080/series/john_doe/securepass123/102451.mkv`

## 9. Legacy M3U Compatibility

---

For primitive hardware profiles lacking an execution parser engine to decode JSON matrices natively, the system exposes an automated translation script via `get.php` to map databases out linearly into text lines.

```
GET http://<base_url>/get.php?
username=<username>&password=<password>&type=m3u_plus&output=ts
```

- `type=m3u_plus`: Signals the core engine to render extended tag headers (including nested attributes like `tvb-id`, `tvb-logo`, and `group-title`).
- `output=ts`: Explicitly instructs live feed lines to output raw MPEG-2 Transport Stream (TS) byte containers instead of complex segmented HLS stream files.